Activity Practice 1: Defining Layers and Responsibilities

1. Defining The Four Layers

Layer	Purpose/Responsibility	Output/Artifact
1. Presentation Layer (UI)	Handles HTTP requests, authentication, session management, and displaying the user interface (views).	Controllers (Example: ProductController)
2. Business Logic Layer (Service/Domain)	Contains core business rules, validation logic, and transaction management. Coordinates data access.	Managers/Services (Example: ProductService)
3. Persistence Layer (Data Access)	Responsible for mapping business objects to database entities and executing CRUD operations (Create, Read, Update, Delete).	Repositories/DAOs (Example: ProductRepository)
4. Data Layer	Physical database storage system (Example: PostgreSQL, MySQL).	Database Schema (Tables)

2. Defining Data Flow

Scenario: The user clicks to view the details of the product "Gaming Laptop" (ID: 101).

- **Step 1 (Request):** Client sends GET /products/101 $\rightarrow$ **Presentation Layer** (ProductController).
- **Step 2:** Layer 1 calls $\rightarrow$ **Business Layer** (ProductService.getDetails(101)).
- **Step 3:** Layer 2 calls $\rightarrow$ **Persistence Layer** (ProductRepository.findById(101)).
- **Step 4:** Layer 3 executes SQL $\rightarrow$ **Data Layer** (SELECT * FROM products WHERE id=101).
- **Step 5 (Data Return):** Data Layer returns ResultSet (raw data) $\rightarrow$ **Persistence Layer**.
- **Step 6:** Layer 3 converts to ProductEntity $\rightarrow$ returns to **Business Layer**.
- **Step 7:** Layer 2 applies logic (checks status = 'active'), converts to ProductDTO (hides import price, only shows selling price) $\rightarrow$ returns to **Presentation Layer**.
- **Step 8 (Response):** Layer 1 packages into JSON $\rightarrow$ Returns to **Client**.

Activity Practice 2: Component Identification

1. Identify Components

- **Layer 1: Presentation**
  - **Component:** ProductController
  - **Responsibility:**
    - Listens to the endpoint GET /api/v1/products/{id}.
    - Extracts {id} from the URL.
    - Calls IProductService.
    - Handles HTTP errors (e.g., returns 404 if the Service reports not found).
- **Layer 2: Business Logic**
  - **Component:** ProductService
  - **Responsibility:**
    - Checks business logic: *If the product is marked "Deleted", throws an exception*.
    - Calls IProductRepository to get raw data.
    - Maps from ProductEntity (DB data) to ProductResponseDTO (API data).
- **Layer 3: Persistence**
  - **Component:** ProductRepository
  - **Responsibility:**
    - Uses JPA/Hibernate or JDBC to connect to the DB.
    - Executes query: SELECT * FROM products WHERE product_id = ?.

2. Define Interfaces

These are contracts between layers to ensure decoupling.

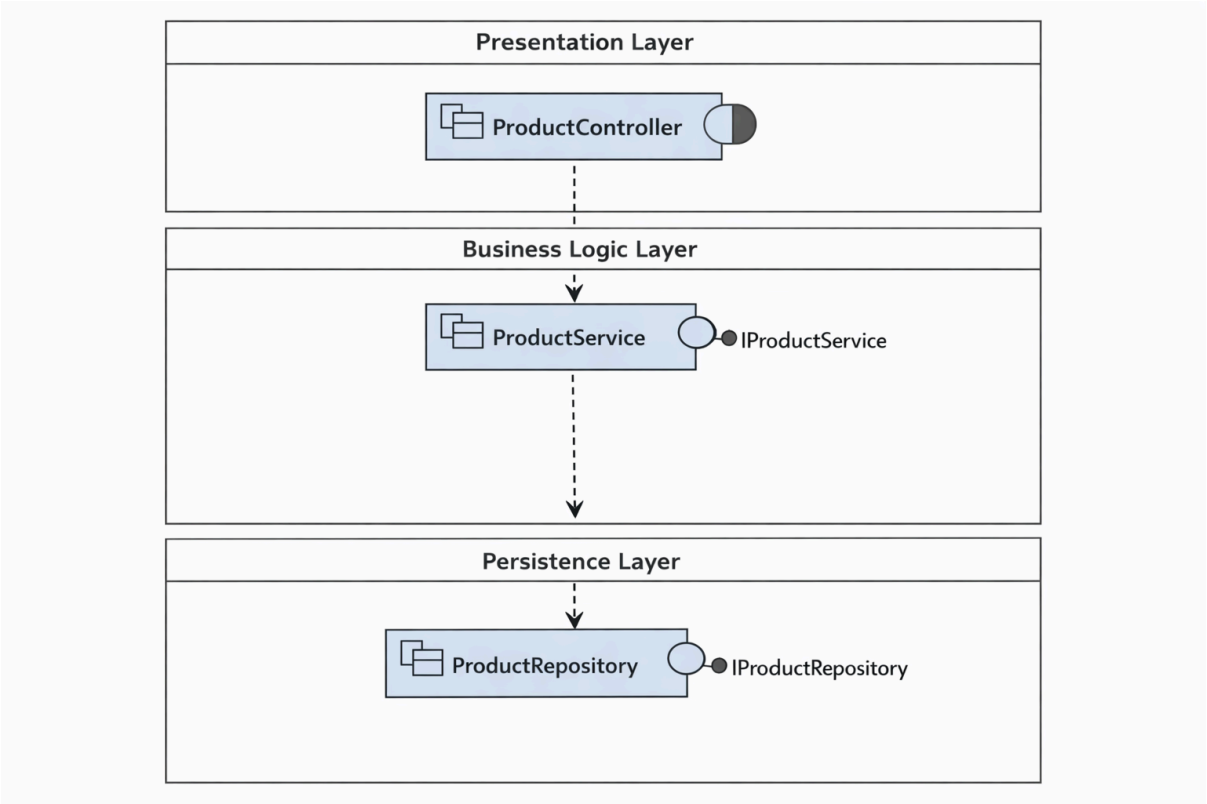
- **Interface for Layer 2 (Service Interface):** Called by the Controller.

```
public interface IProductService { ProductDTO getProductById(Long id);}
```

- **Interface for Layer 3 (Repository Interface):** Called by the Service.

```
public interface IProductRepository { Optional<ProductEntity> findById(Long id);}
```

Activity Practice 3: Component Diagram Modeling (UML)


Presentation Layer
ProductController
Business Logic Layer
ProductService
IProductService
Persistence Layer
ProductRepository
IProductRepository